

MULTI-AGENT REINFORCEMENT LEARNING

Final Project

Policy-Space Response Oracles (PSRO)

Population-Based Learning, Meta-Games, and Strategic Robustness

Project format	Recommended team	Primary setting
Implementation + experiments + report	2-3 students	Two-player zero-sum game

Core question

Does maintaining and solving a population of policies produce a more robust strategic agent than ordinary self-play under the same training budget?

Course handout and assessment specification

1. Project Overview

In this project, students will implement and evaluate Policy-Space Response Oracles (PSRO), a framework for learning strategic policies in multi-agent environments. Unlike standard MARL methods that train one policy against current opponents, PSRO maintains a growing population of policies for every agent. At each iteration, agents compute approximate best-response policies against mixtures of existing opponent policies, while a meta-solver determines how frequently each policy should be played.

The project should demonstrate how population-based learning can reduce strategic overfitting, discover diverse behaviours, and approximate game-theoretic equilibria.

2. Learning Objectives

- Explain the difference between learning in action space and learning in policy space.
- Implement an empirical meta-game from interactions between learned policies.
- Train approximate best-response policies using reinforcement learning.
- Compute deterministic and mixed-strategy Nash equilibria.
- Implement the complete PSRO training loop.
- Measure exploitability, convergence, policy diversity, and robustness.
- Compare alternative meta-strategy solvers.
- Analyse non-transitive strategic behaviour.
- Explain the relationship between PSRO, fictitious play, self-play, and double-oracle methods.

3. Environment

Select a competitive two-player environment with relatively short episodes. Recommended options include:

- Iterated Rock-Paper-Scissors with partial observations.
- Matching Pennies with history-dependent strategies.
- Kuhn Poker or Leduc Poker.
- Predator-Prey with opposing objectives.
- A small grid-world soccer environment.
- A custom security or pursuit-evasion game.

The primary experiment should use a two-player zero-sum game. General-sum games or games with more than two agents may be used as an extension. The environment must contain enough strategic complexity that a single deterministic policy is insufficient.

4. PSRO Framework

For each player i , maintain a population of policies:

$$\Pi_i^t = \{\pi_i^1, \pi_i^2, \dots, \pi_i^{K_i}\}.$$

The empirical payoff matrix records the expected return for every joint combination of policies. In a two-player game:

$$M_{jk} = E[G_1 \mid \pi_1^j, \pi_2^k].$$

For a zero-sum game, $G_2 = -G_1$. At PSRO iteration t :

1. Evaluate every missing combination of population policies.
2. Construct or update the empirical payoff matrix.
3. Apply a meta-solver to calculate a policy mixture for each player.
4. Train an approximate best response against the opponent's mixture.
5. Add the new best-response policy to the player's population.
6. Repeat until the iteration budget is exhausted or exploitability is sufficiently small.

The approximate best response for player i is:

$$\pi_i^{BR} \approx \arg \max_{\pi_i} E_{\pi_{-i} \sim \sigma_{-i}} [G_i(\pi_i, \pi_{-i})].$$

Here, σ_{-i} is the meta-strategy over the opponent's policy population.

5. Required Implementation

Part A: Environment and Baseline Agents

Implement or adapt the selected environment and create at least three baseline policies:

- A random policy.
- A simple heuristic policy.
- A single-policy self-play learner.

Report the returns of the baseline policies against one another.

Part B: Empirical Meta-Game

Implement a component that:

- Stores the policy population of every player.
- Runs repeated matches for every required joint policy combination.
- Estimates expected return and standard error.
- Constructs the empirical payoff matrix.
- Supports adding policies without recomputing existing entries.

Each payoff estimate must use multiple episodes and multiple random seeds.

Part C: Response Oracle

Implement an approximate best-response oracle using Q-learning, DQN, PPO, Actor-Critic, or Policy Gradient. During each training episode, sample the opponent policy according to its current meta-strategy; the sampled opponent remains fixed for that episode. Students may use an RL library for the oracle but must identify which components were implemented independently.

Part D: Meta-Strategy Solvers

Implement at least three solvers from the following list. At least one must generate a non-uniform stochastic mixture.

Solver	Required behaviour
Uniform mixture	Every policy receives equal probability.
Latest-policy self-play	Train only against the newest opponent policy.
Fictitious play	Best respond to the empirical distribution of previous policies.
Nash equilibrium solver	Solve the empirical zero-sum game by linear programming.
Replicator dynamics	Update probabilities according to relative payoff.
Projected replicator dynamics	Maintain exploration with a minimum probability threshold.
Regret matching	Construct the mixture from accumulated positive regret.

Part E: Complete PSRO Algorithm

Implement a configurable PSRO loop with parameters for the number of PSRO iterations, oracle-training steps, payoff-evaluation episodes, meta-solver, population size, random seeds, and stopping condition. Store every policy, payoff matrix, and meta-strategy so an interrupted experiment can be resumed.

6. Required Experiments

Experiment 1: Self-Play versus PSRO

- Average return against the final opponent population.
- Worst-case return.
- Exploitability.
- Sensitivity to opponent selection.

- Performance against unseen opponent policies.

Experiment 2: Meta-Solver Comparison

- Exploitability by PSRO iteration.
- Population size by iteration.
- Expected meta-game value.
- Meta-strategy entropy.
- Wall-clock training time.

Experiment 3: Policy Population Analysis

- Visualise the empirical payoff matrix as a heat map.
- Identify dominated policies and non-transitive cycles.
- Report policies with positive equilibrium probability.
- Determine whether new policies add strategic diversity.
- Discuss convergence or continued cycling.

Experiment 4: Approximate-Oracle Quality

- Use at least three oracle-training budgets.
- Measure effects on convergence, exploitability, stability, population size, and cost.

Experiment 5: Generalisation

- Evaluate against random and heuristic policies.
- Evaluate against earlier population policies and final self-play.
- Evaluate against previously unseen or perturbed opponents.

7. Evaluation Measures

Expected Joint Return

$$V_1(\sigma_1, \sigma_2) = \sigma_1^T M \sigma_2.$$

Best-Response Value

$$BR_1(\sigma_2) = \max_{\pi_1} V_1(\pi_1, \sigma_2).$$

Exploitability

For a two-player zero-sum game:

$$\text{Exploitability}(\sigma) = [BR_1(\sigma_2) - V_1(\sigma_1, \sigma_2)] + [BR_2(\sigma_1) - V_2(\sigma_1, \sigma_2)].$$

If an exact best response is unavailable, report approximate exploitability and explain how it was estimated.

Strategy Entropy

$$H(\sigma_i) = -\sum_k \sigma_i(k) \log \sigma_i(k).$$

Entropy indicates whether the solution depends on one dominant policy or a diverse mixture.

Policy Diversity

Define at least one behavioural measure: action-distribution distance, state-visitation distance, pairwise Jensen-Shannon divergence, or differences in returns against a fixed opponent set.

8. Ablation Study

Perform at least two ablations:

- Remove the population and use only the latest policy.
- Replace the learned meta-strategy with a uniform mixture.
- Reduce the number of payoff-evaluation episodes.
- Always train against one opponent instead of sampling.
- Use a weak response oracle.
- Remove dominated policies.
- Change the initial population.

Explain which PSRO components are responsible for robustness and convergence.

9. Deliverables

- Source code with installation and execution instructions.
- Configuration files for every reported experiment.
- Saved payoff matrices and meta-strategies.
- Training and evaluation scripts.
- A reproducible notebook demonstrating one complete PSRO run.
- A report of up to eight pages.
- A 10-12 minute presentation.
- A short individual or group defence.

The report must include the problem formulation, environment and rewards, PSRO architecture, response oracle, meta-solvers, experimental methodology, confidence intervals, ablations, limitations, possible improvements, and division of work.

10. Grading Rubric

Component	Weight
-----------	--------

Component	Weight
Correct empirical meta-game implementation	15%
Response-oracle implementation	15%
PSRO loop and population management	20%
Meta-strategy solvers	15%
Experimental design and reproducibility	15%
Exploitability and diversity analysis	10%
Report, code quality, and presentation	10%

A project that reports only average reward, without analysing exploitability or strategic robustness, cannot receive full credit.

11. Bonus Extensions

Up to 10 bonus points may be awarded for one substantial extension:

- General-sum PSRO.
- More than two agents.
- Neural population-based training.
- AlphaRank as a meta-solver or evaluation method.
- Rectified PSRO, Pipeline PSRO, or Anytime PSRO.
- Policy distillation or population pruning.
- Opponent modelling.
- Comparison with NFSP or league-based training.
- Statistical testing across five or more seeds.

12. Expected Conclusion

The final discussion must answer the following question:

Does maintaining and solving a population of policies produce a more robust strategic agent than ordinary self-play under the same training budget?

Support the answer using payoff matrices, approximate exploitability, policy mixtures, generalisation results, and behavioural evidence—not only training reward.